Módulo 10 - Aulas 3 e 4

Tarefas

No final deste módulo você deve submeter em um ÚNICO arquivo PDF os seguintes prints:

- Atividade 10.6: passo 7.
- Atividade 10.7: passo 9.
- Atividade 10.8: passo 8.
- Atividade 10.9: passo 12.
- Atividade 10.10: passo 12.

Regras para a elaboração do documento:

1. Antes de cada Print, adicione obrigatoriamente uma frase explicativa que sinalize do que se trata o print. A inserção de prints sem a devida frase explicativa será considerada como tentativa de atrapalhar a correção do instrutor e será penalizada a critério do instrutor. Exemplo:

Print da atividade 10.6: [Imagem com o Print]

2. Os Prints devem ser tirados da TELA CHEIA. Quando tirados da VM da AWS, devem ser capturados **obrigatoriamente** da tela cheia clicando no botão "Screenshot" → "Take screenshot" da barra de ferramentas do Hypervisor da AWS.
3. Insira **somente a quantidade de Prints solicitados por atividade** usando exclusivamente 1 página por print. **A página do documento onde você vai inserir o Print deve estar com a orientação no modo PAISAGEM** ´para termos melhor aproveitamento do espaço. Ou seja, seu documento deverá ter a mesma quantidade de páginas que a quantidade do total de Prints! A inserção de prints desnecessários será considerada como tentativa de atrapalhar a correção do instrutor e será penalizada com nota 0.

4. Apresente Prints legíveis e com tamanho correto para fácil leitura. O envio de prints com letras minúsculas poderá ser considerado como tentativa de atrapalhar a correção do instrutor e será penalizada a critério do instrutor.

Atividade 10.6 – Explorando um Hypervisor Tipo 2 no Kali Linux

Nesta atividade, vamos usar o Kali Linux para explorar o Virtual Box, que é um Hypervisor Tipo 2.

Vamos começar inicializando nosso Kali Linux via RDP ao IP: 192.168.98.40, com usuário “aluno” e senha “rnpesr”.

1. Na barra de tarefas superior, clique no botão “Aplicativos” → “Aplicações habituais” → “Sistema” → “Oracle VM Virtualbox”.
2. Veja que abriu uma janela nova com o VirtualBox. Clique em “Arquivo” → “Preferências...”.
3. Na nova janela aberta, veja que na aba “Geral”, a pasta padrão em “Pasta Padrão para Máquinas” é:

```
/home/aluno/VirtualBox VMs
```

4. A Biblioteca de Autenticação VRDP (VirtualBox Remote Desktop Protocol) é um componente do VirtualBox que fornece mecanismos de autenticação para conexões de desktop remoto com máquinas virtuais do VirtualBox. Ela permite que você se conecte e controle de forma segura a interface de desktop de uma máquina virtual remotamente pela rede. A opção padrão selecionada é “VBoxAuth”.
5. Na coluna da esquerda, clique em “Entrada” e na aba da direita em “Gerenciador do VirtualBox”, veja os comandos de Gerenciamento de VirtualBox e seus atalhos.
6. Agora clique na aba “Máquina Virtual” e veja os comandos de Máquina Virtual e seus atalhos.

7. Na coluna da esquerda, clique em “Idioma” e na aba da direita veja os idiomas disponíveis (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**). Note que está em “Padrão”, isso pega o idioma do sistema operacional, neste caso o português.
8. Na coluna da esquerda, clique em “Tela” e na aba da direita veja os ajustes da interface gráfico que podem ser feitos:

- Tamanho Máximo da Tela do Sistema Convidado
- Fator de Escalonamento
- Recursos Estendidos
- Escalonamento de Fontes

9. Clique em “OK” para fechar a janela “VirtualBox - Preferências”. Não saia do VirtualBox, continuaremos na próxima Atividade.

Parabéns! Agora você conhece as principais abas do VirtualBox!

Atividade 10.7 – Criando uma VM em um Hypervisor Tipo 2 no Kali Linux

Nesta atividade, vamos usar o Kali Linux para criar uma VM que contenha o Debian netinst no VirtualBox que exploramos na Atividade anterior.

O Debian netinst é uma imagem mínima de instalação que contém apenas o necessário para iniciar a instalação e baixar os pacotes adicionais durante o processo. Isso permite que você selecione e instale apenas os componentes necessários para o seu sistema, resultando em uma distribuição Debian mais leve e personalizada. Vamos continuar a Atividade anterior que já tem o VirtualBox aberto.

1. Volte ao VirtualBox, feche a janela que conta com a mensagem “Não foi possível enumerar os dispositivos USB...” e clique em “Novo”.
2. Insira o nome “DebianNetinst”.
3. No campo “Imagem ISO”, clique em “Outro” e na coluna da esquerda navegue pelas pastas “Outros locais” → “Computador” → “curso” → selecione a imagem baixada com 2 cliques “debian-12.5.0-amd64-netinst.iso”.
4. Clique em “Próximo(N)”.

5. No campo “Nome do Usuário”, mude o nome para “DebianNetinst”, visualize que a senha padrão é “changeme” e clique em “Próximo(N)”.
6. Aloque 512MB de “Memória Base” e clique em “Próximo(N)”.
7. Na próxima janela, modifique o “Tamanho do Disco” para 10.22 GB e clique em “Próximo(N)”.
8. Leia todo o sumário que aparece e clique em “Finalizar”.
9. Veja que uma máquina virtual chamada “DebianNetinst” foi criada na coluna da esquerda (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**). O ambiente de VM da AWS não possui driver de virtualização. Em outras palavras, não há necessidade de rodar o DebianNetinst na VM, caso contrário aparecerá um erro de "Kernel driver not installed".
10. Pronto! Agora você criou uma máquina virtual que rodará a imagem do Debian Netinst para ser instalado! Você pode reproduzir esses passos para fazer essa instalação no seu computador pessoal!
11. Feche todas as janelas, encerre a conexão RDP e interrompa a sessão da VM da AWS porque na próxima atividade não será utilizada.

Parabéns! Agora você sabe como adicionar uma imagem de Sistema Operacional para que seja usado no VirtualBox!

Atividade 10.8 – Explorando uma UEFI/BIOS por simulação

Nesta atividade, vamos usar o navegador Mozilla Firefox no seu computador pessoal para acessar um recurso da Lenovo que permite usar dezenas de UEFI/BIOS dos equipamentos da Lenovo de forma simulada.

1. No seu computador pessoal (não use a VM da AWS), abra o seu navegador de preferência e acesse o seguinte dite:

```
https://download.lenovo.com/bsco/index.html#/
```

2. Clique em “Launch Simulator” e selecione o aparelho “Laptops”.
3. Selecione “Lenovo Laptops”.
4. Selecione “Lenovo Slim Pro 9 16IRP8 (83C0)” → “Graphics Interface Mode”.

5. Veja que agora a BIOS foi inicializada de forma simulada. Veja as informações do campo “Information”. Aqui temos:

- Nome do produto
- Versão da BIOS
- CPU
- Memória do sistema
- Secure Boot

6. Clique no campo da esquerda “Configuration”. Veja a configuração do sistema, como o tempo, data, rede WLAN, Graphics Device, Intel(R) Virtualization Technology, Always On USB, Charge in Battery Mode, System Performance Mode, etc.
7. Clique no campo da esquerda “Security”. Veja a configuração de segurança da BIOS: Set Hard Disk Password, Intel Platform Trust Technology, Device Guard, Secure Boot, Reset to Setup Mode e Restore Factory Keys.
8. Clique no campo da esquerda “Boot”. Veja a configuração de Boot da BIOS: USB Boot, PXE Boot to LAN, IPV4 PXE First, EFI, Windows Boot Manager e EFI PXE Network (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**).
9. Finalmente, clique em “Exit” → “Exit Discarding Changes” para sair sem salvar mudanças e clique em “Yes”. Feche o navegador.

Parabéns! Nesta atividade você aprendeu os principais campos de uma BIOS/UEFI com interface gráfica. No mundo real, quase todos os componentes que exploramos estarão presentes na UEFI do seu computador.

Atividade 10.9 – Implementação do Full Disk Encryption em uma partição no Kali Linux

Nesta atividade, vamos usar o Kali Linux para implementar o FDE em uma partição de 200M que criaremos no Kali Linux. Vamos dar continuidade a Atividade anterior.

Vamos começar inicializando nosso Kali Linux via RDP ao IP: 192.168.98.40, com usuário “aluno” e senha “rnpesr”.

1. Abra o Terminal e execute o seguinte comando (com senha “rnpesr”) para ser super usuário:

```
┌──(aluno㉿kali)-[~]
└─$ sudo -i
[sudo] senha para aluno:
```

2. Vamos explorar o armazenamento na nossa máquina virtual (o tamanho dos discos pode mudar um pouco). Escreva o seguinte comando:

```
Disk /dev/nvme0n1: 22 GiB, 23622320128 bytes, 46137344 sectors
Disk model: Amazon Elastic Block Store
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 4096 bytes / 4096 bytes
Disklabel type: gpt
Disk identifier: 9FFA977A-4F2F-44A0-BCAC-15A77352798A

Device           Start      End  Sectors  Size Type
/dev/nvme0n1p1  262144 46137310 45875167 21,9G Linux filesystem
/dev/nvme0n1p14   2048     8191     6144    3M BIOS boot
/dev/nvme0n1p15   8192   262143   253952  124M EFI System

Partition table entries are not in disk order.


Disk /dev/nvme1n1: 2 GiB, 2147483648 bytes, 4194304 sectors
Disk model: Amazon Elastic Block Store
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 4096 bytes / 4096 bytes
Disklabel type: dos
Disk identifier: 0xf8a2dbef

Device         Boot  Start    End Sectors  Size Id Type
/dev/nvme1n1p1        2048 206847  204800  100M 83 Linux
/dev/nvme1n1p2      206848 411647  204800  100M 83 Linux
```

3. Trabalharemos no disco /dev/nvme1n1. Veja que o seguinte comando mostra as partições presentes no nosso disco:

```
┌──(root㉿kali)-[~]
└─# fdisk /dev/nvme1n1

Welcome to fdisk (util-linux 2.41.2).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.


Command (m for help): p
Disk /dev/nvme1n1: 2 GiB, 2147483648 bytes, 4194304 sectors
Disk model: Amazon Elastic Block Store
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 4096 bytes / 4096 bytes
Disklabel type: dos
Disk identifier: 0xf8a2dbef

Device         Boot  Start    End Sectors  Size Id Type
/dev/nvme1n1p1        2048 206847  204800  100M 83 Linux
/dev/nvme1n1p2      206848 411647  204800  100M 83 Linux
```

4. Agora vamos criar a partição nvme1n1p3 com 200M para aplicar o FDE nela:

```
Command (m for help): n
Partition type
   p   primary (2 primary, 0 extended, 2 free)
   e   extended (container for logical partitions)
Select (default p):

Using default response p.
Partition number (3,4, default 3):
First sector (411648-4194303, default 411648):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (411648-4194303, default
4194303): +200M

Created a new partition 3 of type 'Linux' and of size 200 MiB.
```

5. Veja como ficaram as partições e salve as configurações:

```
Command (m for help): p
Disk /dev/nvme1n1: 2 GiB, 2147483648 bytes, 4194304 sectors
Disk model: Amazon Elastic Block Store
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 4096 bytes / 4096 bytes
Disklabel type: dos
Disk identifier: 0x407f5a73

Device         Boot  Start    End Sectors  Size Id Type
/dev/nvme1n1p1        2048 206847  204800  100M 83 Linux
/dev/nvme1n1p2      206848 411647  204800  100M 83 Linux
/dev/nvme1n1p3      411648 821247  409600  200M 83 Linux

Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.

┌──(root㉿kali)-[~]
└─# partprobe /dev/nvme1n1
```

6. Execute o seguinte comando para formatar e criptografar a partição nvme1n1p3. Use a senha “abcd1234”:

```
┌──(root㉿kali)-[~]
└─# cryptsetup luksFormat /dev/nvme1n1p3

WARNING!
========
Isto vai sobrescrever dados em /dev/nvme1n1p3 permanentemente.

Are you sure? (Type 'yes' in capital letters): YES
Digite a senha para /dev/nvme1n1p3:
Verificar senha:
```

7. Desbloqueie a partição e atribua o nome “Teste” à partição digitando o seguinte comando. Use a senha “abcd1234”:

```
┌──(root㉿kali)-[~]
└─# cryptsetup luksOpen /dev/nvme1n1p3 Teste
Digite a senha para /dev/nvme1n1p3:
```

8. Agora você tem uma partição criptografada aberta e pronta para ser formatada com um sistema de arquivos. Use o seguinte comando para formatar com ext4 a partição “Teste”:

```
┌──(root㉿kali)-[~]
└─# mkfs.ext4 /dev/mapper/Teste
mke2fs 1.47.2 (1-Jan-2025)
Creating filesystem with 188416 1k blocks and 47104 inodes
Filesystem UUID: 5a8348f9-b7d0-4cf6-8d4d-7dd8e59fe7f0
Superblock backups stored on blocks:
        8193, 24577, 40961, 57345, 73729

Allocating group tables: done
Writing inode tables: done
Creating journal (4096 blocks): done
Writing superblocks and filesystem accounting information: done
```

9. Agora vamos criar uma pasta vazia para montar nossa partição criptografada:

```
┌──(root㉿kali)-[~]
└─# mkdir -m 777 /mnt/encrypted
```

10. Vamos montar nossa partição criptografada para começar a usá-la:

```
┌──(root㉿kali)-[~]
└─# mount /dev/mapper/Teste /mnt/encrypted
```

11. Após executar o comando do passo 10, a execução do FDE foi montada no diretório de montagem especificado (/mnt/encrypted neste exemplo). Agora você pode acessar e utilizar a partição criptografada no Kali Linux por meio desse diretório. Lembre-se de desmontar a partição quando não estiver em uso para garantir a segurança dos seus dados:

```
┌──(root㉿kali)-[~]
└─# umount /mnt/encrypted
```

12. Note que um Acesso Direto (chamado “Volume de 193MB”) foi criado no Desktop e que você pode acessar essa nova partição por esse meio. Clique 2 vezes no ícone para visualizar a pasta no Thuner (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**).

13. Feche o Thuner e o terminal.

Parabéns! Agora você sabe como aplicar o FDE em qualquer partição no Kali Linux!

Atividade 10.10 – Usando o Ubuntu em um Container Docker no Kali Linux

Nesta atividade vamos rodar um Ubuntu em um Container usando Docker no Kali Linux.

Vamos começar inicializando nosso Kali Linux via RDP ao IP: 192.168.98.40, com usuário “aluno” e senha “rnpesr”.

1. Abra o Terminal e execute o seguinte comando (com senha “rnpesr”) para ser super usuário:

```
┌──(aluno㉿kali)-[~]
└─$ sudo -i
[sudo] senha para aluno:
```

2. Abra o Terminal e verifique se o Docker está rodando corretamente, você deve ter a seguinte saída:

```
┌──(root㉿kali)-[~]
└─# docker

Usage:  docker [OPTIONS] COMMAND

A self-sufficient runtime for containers

Common Commands:
  run         Create and run a new container from an image
  exec        Execute a command in a running container
  ps          List containers
  build       Build an image from a Dockerfile
  pull        Download an image from a registry
  push        Upload an image to a registry
  images      List images
  login       Authenticate to a registry
  logout      Log out from a registry
  search      Search Docker Hub for images
  version     Show the Docker version information
  info        Display system-wide information

Management Commands:
  builder     Manage builds
  buildx*     Docker Buildx
  container   Manage containers
  context     Manage contexts
  image       Manage images
  manifest    Manage Docker image manifests and manifest lists
  network     Manage networks
  plugin      Manage plugins
  system      Manage Docker
  trust       Manage trust on Docker images
  volume      Manage volumes

Swarm Commands:
  swarm       Manage Swarm

Commands:
  attach      Attach local standard input, output, and error streams to
a running container
  commit      Create a new image from a container's changes
  cp          Copy files/folders between a container and the local
filesystem
  create      Create a new container
  diff        Inspect changes to files or directories on a container's
filesystem
  events      Get real time events from the server
```

```
  export      Export a container's filesystem as a tar archive
  history     Show the history of an image
  import      Import the contents from a tarball to create a filesystem
image
  inspect     Return low-level information on Docker objects
  kill        Kill one or more running containers
  load        Load an image from a tar archive or STDIN
  logs        Fetch the logs of a container
  pause       Pause all processes within one or more containers
  port        List port mappings or a specific mapping for the
container
  rename      Rename a container
  restart     Restart one or more containers
  rm          Remove one or more containers
  rmi         Remove one or more images
  save        Save one or more images to a tar archive (streamed to
STDOUT by default)
  start       Start one or more stopped containers
  stats       Display a live stream of container(s) resource usage
statistics
  stop        Stop one or more running containers
  tag         Create a tag TARGET_IMAGE that refers to SOURCE_IMAGE
  top         Display the running processes of a container
  unpause     Unpause all processes within one or more containers
  update      Update configuration of one or more containers
  wait        Block until one or more containers stop, then print their
exit codes

Global Options:
      --config string      Location of client config files (default "/
root/.docker")
  -c, --context string     Name of the context to use to connect to the
daemon (overrides
                           DOCKER_HOST env var and default context set
with "docker context use")
  -D, --debug              Enable debug mode
  -H, --host list          Daemon socket to connect to
  -l, --log-level string   Set the logging level ("debug", "info",
"warn", "error", "fatal")
                           (default "info")
      --tls                Use TLS; implied by --tlsverify
      --tlscacert string   Trust certs signed only by this CA (default
"/root/.docker/ca.pem")
      --tlscert string     Path to TLS certificate file (default "/
root/.docker/cert.pem")
      --tlskey string      Path to TLS key file (default "/
root/.docker/key.pem")
      --tlsverify          Use TLS and verify the remote
```

```
  -v, --version            Print version information and quit

Run 'docker COMMAND --help' for more information on a command.

For more help on how to use Docker, head to https://docs.docker.com/go/
guides/
```

3. Agora vamos ver as imagens que estão disponíveis. Como acabamos de começar a utilização do Docker, deve estar vazia:

```
┌──(root㉿kali)-[~]
└─# docker images
REPOSITORY   TAG       IMAGE ID   CREATED   SIZE
```

4. Em seguida, vamos puxar a imagem do Ubuntu e voltar a listar as imagens disponíveis:

```
┌──(root㉿kali)-[~]
└─# docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
20043066d3d5: Pull complete
Digest:
sha256:c35e29c9450151419d9448b0fd75374fec4fff364a27f176fb458d472dfc9e54
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest
```

5. Veja que agora a imagem do Ubuntu é listada!

```
┌──(root㉿kali)-[~]
└─# docker images
REPOSITORY   TAG       IMAGE ID       CREATED       SIZE
ubuntu       latest    ca2b0f26964c   10 days ago   77.9MB
```

6. Vamos rodar a imagem do Ubuntu. Com o seguinte comando o container é criado, mas não iniciado:

```
┌──(root㉿kali)-[~]
└─# docker run ubuntu
```

7. Para iniciar o container, devemos usar o seguinte comando:

```
┌──(root㉿kali)-[~]
└─# docker run --name MeuContainer -it ubuntu bash
root@2c6580444ba4:/#
```

Aqui, “MeuContainer” é apenas como nós queremos nomear o processo de execução, enquanto “-it ubuntu bash” dá o nome do container que estamos rodando. Perceba que abriu uma conexão SSH e que já estamos executando o Ubuntu!

8. Vamos executar o comando “ls” de listagem de pastas no Ubuntu:

```
┌──(root㉿kali)-[~]
└─# docker run --name MeuContainer -it ubuntu bash
root@2c6580444ba4:/# ls
bin  boot  dev  etc  home  lib  lib32  lib64  libx32  media  mnt  opt
proc  root  run  sbin  srv  sys  tmp  usr  var
```

9. Em paralelo, abra um segundo Terminal no Kali Linux, repita o passo 1 e verifique o principal processo do conteiner:

```
┌──(aluno㉿kali)-[~]
└─$ sudo -i
[sudo] senha para aluno:

┌──(root㉿kali)-[~]
└─# docker top MeuContainer
UID                 PID                 PPID                C
STIME               TTY                 TIME                CMD
root                40370               40342               0
16:09               pts/0               00:00:00            bash
```

10. Ainda no segundo Terminal, vamos ver as estatísticas do container (em tempo real) como utilização de CPU, uso de memória, etc. Escreva “Ctrl + c” para sair após ver as informações:

```
┌──(root㉿kali)-[~]
└─# docker stats

CONTAINER ID   NAME           CPU %     MEM USAGE / LIMIT   MEM %
NET I/O       BLOCK I/O   PIDS
2c6580444ba4   MeuContainer   0.01%     916KiB / 1.926GiB   0.05%
1.16kB / 0B   0B / 0B     1
```

11. Feche o segundo Terminal. Volte ao primeiro Terminal, onde está rodando o Ubuntu. Saia do Ubuntu:

```
root@2c6580444ba4:/# exit
exit
```

12. Agora vamos parar a execução do Container (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**):

```
┌──(root㉿kali)-[~]
└─# docker stop MeuContainer
MeuContainer
```

13. Finalmente, vamos a remover os Containers inativos, como o Ubuntu:

```
┌──(root㉿kali)-[~]
└─# docker container prune
WARNING! This will remove all stopped containers.
Are you sure you want to continue? [y/N] y
Deleted Containers:
2c6580444ba45dcc6ca52159ef009a52f255b63b612cd2a2075ba0a7c1e5f690
fab9028be5b9edd1614c872ee8cb59877b70e51c1dee2c1f28969a08ea24fd81

Total reclaimed space: 8B
```

14. Feche o Terminal.

Parabéns! Agora você conhece os princípios básicos do Docker no Kali Linux!